



Concepts for integer types, not integral types

Document number:

[P3701R0](#)

Date:

2025-05-19

Audience:

CWG, LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-To:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P3701R0/github

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/signed-or-unsigned.cow



ABSTRACT: The `std::integral` concept includes cv-qualified types, character types, and `bool`, which is overly broad for many uses cases. More restrictive concepts are introduced, and editorial changes are made so "integral" and "integer" are distinct terms.

§ Contents

- 1 Introduction
- 2 Design
 - 2.1 Is this obsoleted by P3003?
 - 2.2 Should `const int` be an integer type?
- 3 Impact on the standard
- 4 Wording
 - 4.1 Core wording
 - 4.2 Library wording
- 5 References

§ 1. Introduction



`std::integral` is widely used by the C++ community as a constraint for numeric code ([GitHubSearch]). Many of these uses are questionable because `std::integral` is satisfied by character types such as `char`, as well as `bool`, and cv-qualified versions of all these types. `std::signed_integral` and `std::unsigned_integral` are similarly permissive.



BUG: The constraints on the following template most likely don't match the intent of the author.

```
template <std::integral T>
T add_integers(T x, T y) { return x + y; }
// ...
add_integers(true, true);      // OK?!
add_integers('a', 'b');       // OK?!
add_integers<const int>(1, 2); // OK?!
```

The C++ standard uses "signed or unsigned integer type" as a more appropriate constraint in [numeric.sat.func], [mdspan.extents.overview], and various other subclauses. That name is clunky and needs to be used instead of the obvious "integer type" because "integer" is already synonymous with "integral", and this proposal cleans that up.

For integer types (with new meaning), signed integer types, and unsigned integer types, I propose the corresponding concepts `std::integer`, `std::signed_integer`, and `std::unsigned_integer`. The goal is mainly to provide a simple alternative to the `integral` concepts, which provide more appropriate constraints in user code.

§ 2. Design

The C++ standard currently uses "integer" and "integral" in a surprising way. `std::signed_integral` includes types that are not signed integers, `std::unsigned_integral` includes types that are not unsigned integers, but "integer type" is a synonym for "integral type" ([basic.fundamental] definition of "integer type").

The really confusing part is that "integral" is only sometimes, but not always, a broader term than "integer". This can be improved with the following strategy:

Defined term (if any)	concept	Meaning
integral type	<code>std::integral</code>	Lots of stuff: <code>int</code> , <code>char</code> , <code>const bool</code> , ...
N/A	<code>std::signed_integral</code>	integral types which satisfy <code>std::is_signed_v</code>
N/A	<code>std::unsigned_integral</code>	integral types which do not satisfy <code>std::is_signed_v</code>
integer type	<code>std::integer</code>	synonym for integral type

<u>integer type</u>	<u>std::integer</u>	<u>signed or unsigned integer types</u>
signed integer type	<u>std::signed integer</u>	signed char, signed short, signed int, etc. (also extended signed integers)
unsigned integer type	<u>std::unsigned integer</u>	unsigned char, unsigned short, unsigned int, etc. (also extended unsigned integers)



NOTE: With this new strategy, `char` and `bool` are not integer types, but integral types.

Some alternative designs instead of `std::integer` were considered:

- `std::signed_or_unsigned` would be inappropriate because `std::is_signed_v<float>` is `true`.
- `std::signed_or_unsigned_integer` is comically verbose. It would also perpetuate the "signed or unsigned integer" workaround in wording; we ought to have a simpler name.
- `std::real_integer`, `std::actual_integer`, `std::integral2`, and other such "integers but like, for real this time" names aren't any less confusing than `std::integer`.

§ 2.1. Is this obsoleted by P3003?

[P3003] goes into a different direction: creating a whole library of numeric concepts, which would also include user-defined types (via opt-in).

However, a lot of numeric user code would not be robust enough to take *any type* that behaves like an integer mathematically. User code that is currently constrained with `std::integral<T>` likely makes some assumptions about `T`, like

- expressions of type `T` can be used in a `switch`, as an array index, etc.,
- `T` is trivially copyable and usually small,
- `T` is default-constructible,
- no operation can throw, terminate, etc.,
- `T` can be converted to floating-point types, among other things,
- ...

`std::integer` is a quick way to constrain a function while documenting these assumptions, which is needed in practice.

§ 2.2. Should `const int` be an integer type?

Unlike `std::floating_point` and `std::integral`, `std::integer` (and "integer type") would not include cv-qualified types. Surprisingly, `const int` is not a standard integer or signed integer type, but `const float` is a standard floating-point type. There is an obvious inconsistency here, and we need to decide on a direction.

I argue that `const int` should not be an integer type. While that may be counter-intuitive at first glance, this behavior is tremendously more useful:



- A substantial amount of wording (§4. Wording) can be simplified to just "is of integer type" or "models integer". This would need to be qualified with "cv-unqualified" otherwise.
- When the user constrains a numeric function template so it only accepts integer types, they are not interested in supporting `const int`. Neither is the standard library. If the template parameter is deduced from a `T x` function parameter, the only way to provide `T = const int` is explicitly anyway; it never happens organically through deduction. In the extremely unlikely event they want to support it, they should do so explicitly via `std::remove_cv_t`.
- It is *extremely* unlikely that the user intended to support `volatile int` in their types and functions, so this should require opt-in, not opt-out. `std::integer<std::remove_volatile_t<T>>` would be opt-in. Opt-out would be the natural consequence of including cv-qualified types in "integer type".
- Every occurrence of "signed integer type" and "unsigned integer type" should not include cv-qualified types. For example, it would be insane if the `size_type` of a container could be `const`.
- The standard needs to say "cv-unqualified floating-point type(s)" in a number of places ([N5008]), and when it doesn't say "cv-unqualified", it's not always obvious whether that is accidental or by intention. For example, [basic.extended.fp] never says "cv-unqualified", but we obviously don't want `using float32_t = const _Float32;`. Generally, including cv-qualified types in the definitions by default may be more concise in some places, but it's making the standard less expressive; it leaves intent unexpressed.

In conclusion, `const int` being considered an integer may feel philosophically right, but it's not very useful in wording, and it's not beneficial to `std::integer` constraints. `const float` being a floating-point type is a bad status quo that we should not perpetuate.

§ 3. Impact on the standard

Firstly, the three new concepts `std::integer`, `std::signed_integer`, and `std::unsigned_integer` are added.

Secondly, we make editorial changes so that "integer" is consistently a narrower term, and "integral" is a broader term. See §2. Design.

While the changes are all very simple, the wording impact is quite immense because "integer" and "integral" have been used interchangeably throughout wording. Any existing uses of "integer" are replaced with "integral", and the "signed or unsigned integer" pattern can be simplified to just "integer".

§ 4. Wording

The following changes are relative to [N5008].



§ 4.1. Core wording

Change [basic.fundamental] paragraph 5 as follows:



Each value x of an unsigned integer type with width N has a unique representation $x = x_0 2^0 + x_1 2^1 + \dots + x_{N-1} 2^{N-1}$, where each coefficient x_i is either 0 or 1; this is called the *base-2 representation* of x . The base-2 representation of a value of signed integer type is the base-2 representation of the congruent value of the corresponding unsigned integer type. The standard signed integer types and standard unsigned integer types are collectively called the *standard integer types*, and the extended signed integer types and extended unsigned integer types are collectively called the *extended integer types*. The standard integer types and extended integer types are collectively called integer types.

[Note: The set of integer types is equal to the set of signed and unsigned integer types. — end note]



NOTE: If you cannot see the mathematical symbols in the paragraph above, you are viewing this document in an old browser with no MathML support.

Change [basic.fundamental] paragraph 6 as follows:



A fundamental type specified to have ~~a signed or unsigned~~ an integer type as its *underlying type* has the same object representation, value representation, alignment requirements ([basic.align]), and range of representable values as the underlying type. Further, each value has the same representation in both types.

Change [basic.fundamental] paragraph 8 as follows:



Type `wchar_t` is a distinct type that has an implementation-defined ~~signed or unsigned~~ integer type as its underlying type.

Change [basic.fundamental] paragraph 11 as follows:



The types `char`, `wchar_t`, `char8_t`, `char16_t`, and `char32_t` are collectively called *character types*. The character types, integer types, `bool`, ~~the signed and unsigned integer types~~, and cv-qualified versions ([basic.type.qualifier]) thereof, are collectively ~~termed~~ called *integral types*. ~~A synonym for integral type is integer type.~~

Change [conv.rank] paragraph 1 as follows:



Every ~~integer type~~ integral type has an ~~integer~~ integral conversion rank defined as follows:

- [...]

- The rank of any extended signed integer type relative to another extended signed integer type with the same width is implementation-defined, but still subject to the other rules for determining the **integer integral** conversion rank.
- For all **integer integral** types T1, T2, and T3, if T1 has greater rank than T2, and T2 has greater rank than T3, then T1 has greater rank than T3.

[Note: The **integer integral** conversion rank is used in the definition of the integral promotion ([conv.prom]) and the usual arithmetic conversions ([expr.arith.conv]). — end note]



NOTE: Notice that a comma is inserted within the last bullet.

Change [conv.lval] paragraph 3, bullet 5 as follows:



Otherwise, the object indicated by the glvalue is read ([defs.access]). Let V be the value contained in the object. If T is an **integer integral** type, the prvalue result is the value of type T congruent ([basic.fundamental]) to V, and V otherwise.

Change [conv.prom] paragraph 2 as follows:



A prvalue that is not a converted bit-field and has an **integer integral** type other than `bool`, `char8_t`, `char16_t`, `char32_t`, or `wchar_t` whose **integer integral** conversion rank ([conv.rank]) is less than the rank of `int` can be converted to a prvalue of type `int` if `int` can represent all the values of the source type; otherwise, the source prvalue can be converted to a prvalue of type `unsigned int`.

Change [conv.prom] paragraph 3 as follows:



A prvalue of an unscoped enumeration type whose underlying type is not fixed can be converted to a prvalue of the first of the following types that can represent all the values of the enumeration ([dcl.enum]): `int`, `unsigned int`, `long int`, `unsigned long int`, `long long int`, or `unsigned long long int`. If none of the types in that list can represent all the values of the enumeration, a prvalue of an unscoped enumeration type can be converted to a prvalue of the extended integer type with lowest **integer integral** conversion rank ([conv.rank]) greater than the rank of `long long` in which all the values of the enumeration can be represented. If there are two such extended types, the signed one is chosen.

Change [conv.integral] paragraph 1 as follows:



A prvalue of an **integer integral** type can be converted to a prvalue of another **integer integral** type. A prvalue of an unscoped enumeration type can be converted to a prvalue of an **integer integral** type.

Change [conv.fpint] paragraph 1 as follows:



A prvalue of a floating-point type can be converted to a prvalue of an **integer integral** type. The conversion truncates; that is, the fractional part is discarded. The behavior is undefined if the truncated value cannot be represented in the destination type.

Change [\[conv.fpint\]](#) paragraph 2 as follows:



A prvalue of an **integer integral** type or of an unscoped enumeration type can be converted to a prvalue of a floating-point type. [...]

Change [\[expr.reinterpret.cast\]](#) paragraph 5 as follows:



A value of integral type or enumeration type can be explicitly converted to a pointer. A pointer converted to an **integer integral type** of sufficient size (if any such exists on the implementation) and back to the same pointer type will have its original value ([\[basic.compound\]](#)); mappings between pointers and **integers integral types** are otherwise implementation-defined.

Change [\[expr.assign\]](#) paragraph 2 as follows:



In simple assignment (`=`), let *V* be the result of the right operand; the object referred to by the left operand is modified ([\[defns.access\]](#)) by replacing its value with *V* or, if the object is of **integer integral** type, with the value congruent ([\[basic.fundamental\]](#)) to *V*.

Change [\[dcl.init.list\]](#) paragraph 7 as follows:



A *narrowing conversion* is an implicit conversion

- from a floating-point type to an **integer integral** type, or
- [...]
- from an **integer integral** type or unscoped enumeration type to a floating-point type, except where the source is a constant expression and the actual value after conversion will fit into the target type and will produce the original value when converted back to the original type, or
- from an **integer integral** type or unscoped enumeration type to an **integer integral** type that cannot represent all the values of the original type, except where [...]
- [...]

Change [\[dcl.enum\]](#) paragraph 11 as follows:



The value of an enumerator or an object of an unscoped enumeration type is converted to an **integer integral type** by integral promotion ([\[conv.prom\]](#)).

Change [\[class.bit\]](#) paragraph 4 as follows:



If a value of integral type (other than `bool`) is stored into a bit-field of width N and the value would be representable in a hypothetical ~~signed or unsigned~~ integer type with width N and the same signedness as the bit-field's type, the original value and the value of the bit-field compare equal. [...]



§ 4.2. Library wording

Change [\[bitmask.types\] paragraph 1](#) as follows:



Several types defined in [\[support\]](#) through [\[exec\]](#) and [\[depr\]](#) are bitmask types. Each bitmask type can be implemented as an enumerated type that overloads certain operators, as an ~~integer~~ integral type, or as a bitset.

Change [\[support.types.layout\]](#) as follows:



Recommended practice: An implementation should choose types for `ptrdiff_t` and `size_t` whose ~~integer~~ integral conversion ranks ([\[conv.rank\]](#)) are no greater than that of `signed long int` unless a larger size is necessary to contain all the possible values.

Change [\[version.syn\] paragraph 2](#) as follows:



```
#define __cpp_lib_concepts 202207L 20XXXXL
// freestanding, also in <concepts>, <compare>
```

Change [\[numeric.limits.general\] paragraph 4](#) as follows:



Specializations shall be provided for each arithmetic type, both floating-point and ~~integer~~ integral, including `bool`. The member `is_specialized` shall be `true` for all such specializations of `numeric_limits`.

Change [\[numeric.limits.members\] paragraph 10](#) as follows:



For ~~integer~~ integral types, the number of non-sign bits in the representation.

Change [\[numeric.limits.members\] paragraph 18](#) as follows:



`true` if the type is ~~integer~~ integral.

Change [\[numeric.limits.members\] paragraph 20](#) as follows:



`true` if the type uses an exact representation. All ~~integer~~ integral types are exact, but not all exact types are ~~integer~~ integral. For example, rational and fixed-exponent representations are exact but not ~~integer~~ integral.

Change [\[numeric.limits.members\] paragraph 23](#) as follows:



For **integer integral** types, specifies the base of the representation.¹⁶⁷



Change [\[numeric.limits.members\]](#) paragraph 65 as follows:



Meaningful for all floating-point types. Specializations for **integer integral** types shall return `round_toward_zero`.

Change [\[climits.syn\]](#) note 1 as follows:



[Note: Except for `CHAR_BIT` and `MB_LEN_MAX`, a macro referring to an **integer integral** type `T` defines a constant whose type is the promoted type of `T` ([\[conv.prom\]](#)). — *end note*]

Change [\[cstdint.syn\]](#) paragraph 1 as follows:



The header `<cstdint>` supplies **integer integral** types having specified widths, and macros that specify limits of **integer integral** types.

Change [\[cstdint.syn\]](#) paragraph 3 as follows:



All types that use the placeholder *N* are optional when *N* is not 8, 16, 32, or 64. The exact-width types `intN_t` and `uintN_t` for *N* = 8, 16, 32, and 64 are also optional; however, if an implementation defines **integer integral** types with the corresponding width and no padding bits, it defines the corresponding *typedef-names*. Each of the macros listed in this subclause is defined if and only if the implementation defines the corresponding *typedef-name*.

[Note: The macros `INTN_C` and `UINTN_C` correspond to the *typedef-names* `int_leastN_t` and `uint_leastN_t`, respectively. — *end note*]

Change [\[concepts.syn\]](#) as follows:



```
namespace std {
    [...]
    // [concepts.arithmetic], arithmetic concepts
    template<class T>
        concept integral = see below;
    template<class T>
        concept signed_integral = see below;
    template<class T>
        concept unsigned_integral = see below;
    template<class T>
        concept integer = see below;
    template<class T>
        concept signed_integer = see below;
    template<class T>
        concept unsigned_integer = see below;
    template<class T>
        concept floating_point = see below;
```

```
[...]  
}
```



Change [\[concepts.arithmetic\]](#) as follows:



```
template<class T, class... Us>  
    concept one-of = (same_as<T, Us> || ...); // expositi  
template<class T>  
    concept non-integer = one-of<T, bool, char, wchar_t, // expositi  
                                char8_t, char16_t, char32_t>;  
template<class T>  
    concept cv-unqualified = !is_const_v<T> && !is_volatile_v<T>; // expositi  
  
template<class T>  
    concept integral = is_integral_v<T>;  
template<class T>  
    concept signed_integral = integral<T> && is_signed_v<T>;  
template<class T>  
    concept unsigned_integral = integral<T> && !signed_integral<T>;  
template<class T>  
    concept integer = integral<T> && !non-integer<T> && cv-unqualified<T>;  
template<class T>  
    concept signed_integer = integer<T> && is_signed_v<T>;  
template<class T>  
    concept unsigned_integer = integer<T> && is_unsigned_v<T>;  
template<class T>  
    concept floating_point = is_floating_point_v<T>;
```

[Note: signed_integral can be modeled even by types that are not signed integer types ([\[basic.fundamental\]](#)); for example `char`. signed_integer is modeled exclusively by signed integer types. — end note]

[Note: unsigned_integral can be modeled even by types that are not unsigned integer types ([\[basic.fundamental\]](#)); for example, `bool`. unsigned_integer is modeled exclusively by unsigned integer types. — end note]

Change [\[intseq.intseq\]](#) paragraph 1 as follows:



Mandates: T ~~is an integer type~~ models integral.

Change [\[utility.intcmp\]](#) paragraph 1 and [\[utility.intcmp\]](#) paragraph 4 as follows:



Mandates: Both T and U ~~are standard integer types or extended integer types ([\[basic.fundamental\]](#))~~ model integer.

Change [\[utility.intcmp\]](#) paragraph 9 as follows:



Mandates: Both T and R ~~are standard integer types or extended integer types~~



Change [\[forward.list.ops\]](#) paragraph 1 as follows:



In this subclause, arguments for a template parameter named Predicate or BinaryPredicate shall meet the corresponding requirements in [\[algorithms.requirements\]](#). The semantics of $i + n$, where i is an iterator into the list and n is ~~an integer~~ of integral type, are the same as those of `next( $i$ ,  $n$ )`. The expression $i - n$, where i is an iterator into the list and n is an integer, means an iterator j such that $j + n == i$ is **true**. For merge and sort, the definitions and requirements in [\[alg.sorting\]](#) apply.

Change [\[hive.operations\]](#) paragraph 1 as follows:



In this subclause, arguments for a template parameter named Predicate or BinaryPredicate shall meet the corresponding requirements in [\[algorithms.requirements\]](#). The semantics of $i + n$ and $i - n$, where i is an iterator into the hive and n is ~~an integer~~ of integral type, are the same as those of `next( $i$ ,  $n$ )` and `prev( $i$ ,  $n$ )`, respectively. For sort, the definitions and requirements in [\[alg.sorting\]](#) apply.

Change [\[list.ops\]](#) paragraph 1 as follows:



In this subclause, arguments for a template parameter named Predicate or BinaryPredicate shall meet the corresponding requirements in [\[algorithms.requirements\]](#). The semantics of $i + n$, where i is an iterator into the list and n is ~~an integer~~ of integral type, are the same as those of `next( $i$ ,  $n$ )`. The expression $i - n$, where i is an iterator into the list and n is an integer, means an iterator j such that $j + n == i$ is **true**. For merge and sort, the definitions and requirements in [\[alg.sorting\]](#) apply.

Change [\[mdspan.extents.overview\]](#) paragraph 1 as follows:



Mandates:

- IndexType ~~is a signed or unsigned integer type~~ models integer, and
- each element of Extents is either equal to `dynamic_extent`, or is representable as a value of type IndexType.

Change [\[mdspan.sub.strided.slice\]](#) paragraph 3 as follows:



Mandates: Each of the types OffsetType, ExtentType, and StrideType ~~are signed or unsigned integer types, or~~ models integer or integral-constant-like.



NOTE: The "each of the types" part clarifies the wording in the same style as more

recent wording in [\[numerics.c.ckdint\]](#).



Change [\[mdspan.sub.helpers\]](#) paragraph 1 and [\[mdspan.sub.helpers\]](#) paragraph 10 as follows:



Mandates: `IndexType` ~~is a signed or unsigned integer type~~ models integer.

Change [\[ranges.syn\]](#) paragraph 1 as follows:



Within this Clause, for an integer-like type `X` ([\[iterator.concept.winc\]](#)), *make-unsigned-like-t<X>* denotes `make_unsigned_t<X>` if `X` is an ~~integer~~ integral type; otherwise, it denotes a corresponding unspecified unsigned-integer-like type of the same width as `X`. For an expression `x` of type `X`, *to-unsigned-like(x)* is `x` explicitly converted to *make-unsigned-like-t<X>*.

Change [\[ranges.syn\]](#) paragraph 2 as follows:



Also within this Clause, *make-signed-like-t<X>* for an integer-like type `X` denotes `make_signed_t<X>` if `X` is an ~~integer~~ integral type; otherwise, it denotes a corresponding unspecified signed-integer-like type of the same width as `X`.

Change [\[numeric.ops.gcd\]](#) paragraph 1 and [\[numeric.ops.lcm\]](#) paragraph 1 as follows:



Mandates: `M` and `N` both are ~~integer~~ integral types other than `bool`.

Change [\[numeric.ops.midpoint\]](#) paragraph 1 as follows:



Returns: Half the sum of `a` and `b`. If `T` is an ~~integer~~ integral type and the sum is odd, the result is rounded towards `a`.

In [\[numeric.sat.func\]](#), change paragraphs 2, 4, 6, and 8 as follows:



Constraints: `T` ~~is a signed or unsigned integer type ([basic.fundamental])~~ models integer.

Change [\[numeric.sat.cast\]](#) paragraph 1 as follows:



Constraints: `R` and `T` ~~are signed or unsigned integer types ([basic.fundamental])~~ model integer.

Change [\[charconv.syn\]](#) paragraph 1 as follows:



When a function is specified with a type placeholder of *integer-type*, the implementation provides overloads for `char` and all ~~cv-unqualified signed and unsigned integer~~ integer types in lieu of *integer-type*. When a function is specified with a type placeholder of *floating-point-type*, the implementation provides

overloads for all cv-unqualified floating-point types ([[basic.fundamental](#)]) in lieu of *floating-point-type*.



NOTE: Signed or unsigned integer types are already cv-unqualified; this wording used to be redundant.

Change [[format.string.std](#)] paragraph 10 as follows:



If { *arg-id*_{opt} } is used in a *width* or *precision* option, the value of the corresponding formatting argument is used as the value of the option. The option is valid only if the corresponding formatting argument is of ~~signed or unsigned~~ integer type. If its value is negative, an exception of type `format_error` is thrown.

Change [[cmplx.over](#)] paragraph 2, bullet 2 as follows:



Otherwise, if the argument has ~~integer~~ integral type, then it is effectively cast to `complex<double>`.

Change [[cmplx.over](#)] paragraph 3 as follows:



Function template `pow` has additional `constexpr` overloads sufficient to ensure, for a call with one argument of type `complex<T1>` and the other argument of type `T2` or `complex<T2>`, both arguments are effectively cast to `complex<common_type_t<T1, T3>>`, where `T3` is `double` if `T2` is an ~~integer~~ integral type and `T2` otherwise. If `common_type_t<T1, T3>` is not well-formed, then the program is ill-formed.

Change [[rand.util.seedseq](#)] paragraph 2 as follows:



Constraints: `T` ~~is an integer type~~ models integral.

Change [[rand.util.seedseq](#)] paragraph 4 as follows:



Mandates: `iterator_traits<InputIterator>::value_type` ~~is an integer type~~ models integral.

Change [[numerics.c.ckdint](#)] paragraph 1 as follows:



Mandates: Each of the types `type1`, `type2`, and `type3` ~~is a cv-unqualified signed or unsigned integer type~~ models integer.

Change [[cmath.syn](#)] paragraph 3 as follows:



For each function with at least one parameter of type *floating-point-type* other than `abs`, the implementation also provides additional overloads sufficient to ensure that, if every argument corresponding to a *floating-point-type* parameter has arithmetic type, then every such argument is effectively cast to the floating-point type

with the greatest floating-point conversion rank and greatest floating-point conversion subrank among the types of all such arguments, where arguments of **integer** integral type are considered to have the same floating-point conversion rank as **double**. If no such floating-point type with the greatest rank and subrank exists, then overload resolution does not result in a usable candidate ([\[over.match.general\]](#)) from the overloads provided by the implementation.

Change [\[simd.ctor\]](#) paragraph 7, bullet 2 as follows:



both U and value_type are integral types and the **integer** integral conversion rank ([\[conv.rank\]](#)) of U is greater than the **integer** integral conversion rank of value_type, or

Change the title of [\[atomics.types.int\]](#) as follows:



Specializations for **integers integral types**

§ 5. References

[N5008] *Thomas Köppe*. Working Draft, Programming Languages — C++ (2025-03-15)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf>

[GitHubSearch] GitHub code search for uses of std::integral
<https://github.com/search?q=language%3AC%2B%2B+%2Fstd%3A%3Aintegral%5B+%3C%5D%2F&type=code>

[P3003] *Johel Ernesto Guerrero Pe~na*. The design of a library of number concepts
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p3003r0.pdf>